

ALGORITHME autocompletion(prefixe, contexte, n=3)

ENTREE: prefixe (mot partiel), contexte (phrase), n (ordre n-gram)

SORTIE: liste de suggestions triées

suggestions_combinees = liste_vide()

// Étape 1: Prédiction basée sur le contexte (n-grams)

SI contexte ≠ "" ALORS

 mots_contexte = tokeniser(contexte)

 SI longueur(mots_contexte) >= n-1 ALORS

 contexte_recent = mots_contexte[-(n-1):]

 SI contexte_recent ∈ ngrams ALORS

 predictions_ngram = ngrams[contexte_recent]

 POUR CHAQUE (mot, prob) DANS predictions_ngram FAIRE

 SI mot.startsWith(prefixe) ALORS

 score = prob * 0.7 // Poids pour n-grams

 ajouter(suggestions_combinees, (mot, score, 'ngram'))

 FIN SI

 FIN POUR

 FIN SI

 FIN SI

FIN SI

// Étape 2: Prédiction basée sur le préfixe

SI prefixe ≠ "" ALORS

 mots_vocabulaire = filtrer_par_prefixe(vocabulaire, prefixe)

 POUR CHAQUE mot DANS mots_vocabulaire FAIRE

 SI mot ≠ prefixe ALORS

// Score basé sur fréquence et longueur

freq_score = frequencies[mot] / max_frequence

length_score = longueur(prefixe) / longueur(mot)

score = freq_score * 0.5 + length_score * 0.3

ajouter(suggestions_combinees, (mot, score, 'prefix'))

FIN SI

FIN POUR

FIN SI

// Étape 3: Fallback - mots fréquents

SI suggestions_combinees = ∅ ALORS

mots_frequents = obtenir_mots_frequents(10)

POUR CHAQUE (mot, freq) DANS mots_frequents FAIRE

score = freq / 100

ajouter(suggestions_combinees, (mot, score, 'frequence'))

FIN POUR

FIN SI

// Étape 4: Fusion et tri

suggestions_triees = trier_par_score(suggestions_combinees, ordre='desc')

// Étape 5: Déduplication et limitation

suggestions_finales = dedoublonner(suggestions_triees)

suggestions_finales = limiter_a(suggestions_finales, 5)

RETOURNER suggestions_finales

FIN ALGORITHME

ALGORITHME entrainer_ngrams(corpus, n=3)

ENTREE: corpus (liste de textes), n (ordre)

SORTIE: dictionnaire de n-grams

ngrams = dictionnaire_vide()

POUR CHAQUE texte DANS corpus FAIRE

 mots = tokeniser(texte)

 POUR i = 0 À longueur(mots)-n FAIRE

 contexte = tuple(mots[i:i+n-1])

 mot_suivant = mots[i+n-1]

 SI contexte $\notin$ ngrams ALORS

 ngrams[contexte] = compteur_vide()

 FIN SI

 ngrams[contexte][mot_suivant] += 1

 FIN POUR

FIN POUR

// Normaliser les compteurs en probabilités

POUR CHAQUE contexte DANS ngrams FAIRE

 total = somme(ngrams[contexte].valeurs())

 POUR CHAQUE mot DANS ngrams[contexte] FAIRE

 ngrams[contexte][mot] = ngrams[contexte][mot] / total

 FIN POUR

FIN POUR

RETOURNER ngrams

FIN ALGORITHME